Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MINI PROJECT REPORT

FOUNDATION OF PROGRAMMING

1. TITLE PAGE

Title-Bank Management System

TEAM MEMBERS

- Ashray Shingala- 1262251420
- Dhruv Jadhav -1262252800
- Aaryan Kulkarni –1262252488

Project Guide- Dr. Renuka Amit Mane

Institution Name- MIT World Peace University

Submission Date-

NAME	ROLE	TASK
AARYAN KULKARNI	Core Developer	• `struct Account` design & management • File handling • Utility functions: • Date & time: • Transaction history
DHRUV JADHAV	Customer Module	• `deposit()` • `withdraw()` • `transfer()` • `applyInterest(` • `checkBalance()` • `fullStatement(` • Login logic (PIN attempts + freeze logic in main)
ASHRAY SHINGALA	Admin & Security	• Account management: • `createAccount()` • `deleteAccount()` • Admin controls:

		• viewAll() • totalBankBalance() • freezeAccount() • unfreezeAccount() • changeToSavings() • changeToCurrent() • adminDeposit() • adminWithdraw() • Security: • generateOTP() • verifyOTP() • forgotPIN()

ACKNOWLEDGMENT

I would like to sincerely thank my project guide- Renuka Mane mam for guiding me throughout this project. Their support and suggestions helped me understand the concepts clearly and complete the project successfully.

This project on Bank Management System helped me improve my knowledge of C programming, file handling, and basic banking operations like deposit, withdrawal, transfer, and account management. While developing this project, I learned many new things and gained practical experience.

I am also thankful to my teachers and my friends for encouraging and supporting me during the completion of this project.

ABSTRACT

Objectives

The main objective of this project is to develop a simple Bank Management System using C programming language. The system is designed to perform basic banking operations like creating accounts, depositing money, withdrawing money, transferring funds, and checking account balance.

Another objective is to make the system secure by adding features like PIN authentication, OTP verification, daily withdrawal limit, and account freezing after multiple wrong login attempts.

This project also aims to improve understanding of programming concepts such as structures, file handling, functions, and data management.

Methods

To develop this project, C programming language was used. Structures were used to store account details such as account number, name, balance, PIN, and transaction history.

File handling was implemented to store and retrieve account data permanently using a data file, so that information is not lost after the program closes.

Different functions were created for each operation like deposit, withdraw, transfer, interest calculation, and account management. Security features like OTP generation and login attempt tracking were also implemented to make the system more reliable.

Key Results

After completing the project, the system was able to successfully perform all basic banking operations. Users can create accounts, perform transactions, check balance, and view transaction history without errors.

The security features like OTP verification and account freezing worked properly. The data was saved correctly in the file and remained stored even after restarting the program.

Overall, the system functioned smoothly and achieved the expected results.

Conclusion

In conclusion, the Bank Management System project helped in understanding how banking operations can be managed using programming. It improved practical knowledge of C language and strengthened concepts like structures and file handling.

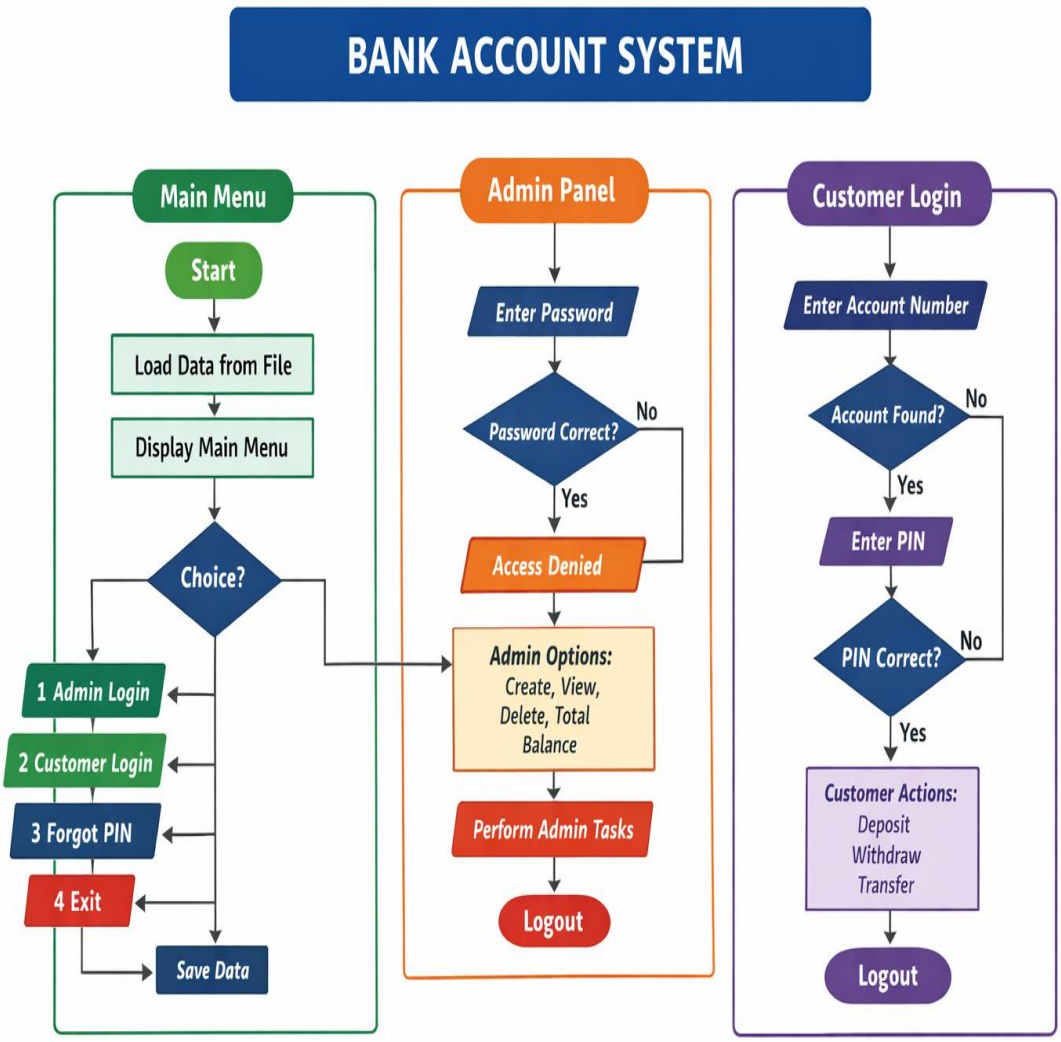
The project shows that even a simple program can handle important tasks like account management and transaction processing when designed properly.

TABLE OF CONTENT

1. Title Page
2. Acknowledgment
3. Abstract
4. Table of Contents
5. List of Figures and Tables
6. Introduction
7. Literature Review

8. System Design
9. Implementation
10. Results
11. Conclusion
12. Future Scope
13. Research Paper Details
14. Plagiarism Check
15. References
16. Appendix

LIST OF FIGURE AND TABLE



Sr no.	System name	Technology Used	Features	performance
1	Core Banking System	C Programming	Account creation, deposit, withdraw, transfer	**medium**
2	Customer Module	C Functions & Logic	Balance check, transfer, daily withdraw limit	**Medium**
3	Admin Module	C Programming	Create/delete account, freeze/unfreeze	**Low**
4	File Storage System	Binary File Handling	Fast data saving/loading, persistent storage	**High**
5	Security System	PIN & Login Attempt Logic	Account freeze after multiple failed attempts	**medium**

INTRODUCTION

PROBLEM STATEMENT- Managing bank accounts and transactions manually can lead to errors, security issues, and difficulty in maintaining proper records. There is a need to develop a simple Bank Management System in C that can securely handle account creation, transactions, and data storage efficiently.

BACKGROUND CONTEXT- The subject *Foundation of Programming in C* teaches the basics of computer programming using the C language. It focuses on understanding how to write, compile, and run programs, and introduces fundamental concepts like variables, data types, loops, and functions. Students also learn about problem-solving techniques and how to design algorithms to perform specific tasks. The course provides a strong foundation for developing software and understanding more advanced programming concepts in the future.

OBJECTIVES

- To understand the basic concepts of programming using the C language.
- To learn how to write, compile, and execute C programs.
- To gain knowledge of data types, variables, operators, loops, and control structures.
- To develop problem-solving skills and the ability to design algorithms.
- To understand functions, arrays, and structures for organized programming.
- To build a strong foundation for learning advanced programming languages and software development.

LITERATURE REVIEW

Sr no.	System	Technology Used	Key Features	Limitations
1	Traditional Banking System	Manual	Record keeping, basic transactions	Time-consuming, high chances of human error
2	ATM System	Embedded Systems	Cash withdrawal, balance inquiry	Limited services, no advanced features
3	Online Banking System	Web Applications	Fund transfer, account management	Requires internet, security risks
4	Mobile Banking Applications	Android / iOS Apps	Easy access, notifications, digital payments	Device dependency, privacy concerns
5	Proposed C Banking System	C Programming + File Handling	Account management, OTP, transaction history	No GUI, limited scalability, basic security

==========

System Design / Methodology

The proposed system is a **Banking Management System developed in C programming language**. It is designed to perform basic banking operations such as account creation, deposit, withdrawal, fund transfer, and account management. The system follows a **modular approach**, where different functionalities are divided into separate modules like Admin, Customer, Security, and File Handling. This improves code organization, readability, and maintainability.

The system is divided into the following main components:

- **Admin Module**: Responsible for managing accounts, including creating, deleting, freezing, and updating account types.
- **Customer Module**: Allows users to perform operations like deposit, withdrawal, transfer, and balance inquiry.
- **Security Module**: Provides authentication using PIN, OTP verification, and account freezing after multiple failed login attempts.

- **File Handling Module**: Manages data storage using binary files to ensure persistence of account information.

METHODOLOGY

The working of the system is based on a step-by-step procedure:

Step 1: Data Initialization
When the program starts, it loads all account data from a binary file. If the file is not available, the system initializes with zero accounts.

Step 2: User Selection
The system displays a menu where the user selects one of the options: Admin, Customer, Forgot PIN, or Exit.

Step 3: Authentication

- Admin logs in usi

- ng a predefined password.
- Customer logs in using account number and PIN.
- OTP verification is used for sensitive operations like account creation and PIN reset.

Step 4: Execution of Operations
After successful login:

- Customers can perform deposit, withdraw (with daily limit), transfer money, check balance, and view transaction history.
- Admin can create or delete accounts, freeze/unfreeze accounts, and view all account details.

Step 5: Data Update and Storage
All operations update the account data, and the changes are saved back into the binary file. Transaction history is also recorded with date and time.

Step 6: Exit Process
Before exiting the program, all data is saved to ensure no loss of information.

IMPLEMENTATION

TOOL USED

- **Programming Language:** C
- **Compiler:** GCC / Turbo C / Code::Blocks
- **Platform:** Windows (Command Line Interface)
- **Libraries Used:**
  - `stdio.h` → Input/Output operations
  - `string.h` → String handling
  - `stdlib.h` → Memory and utility functions
  - `time.h` → Date, time, and OTP generation
- **Storage:** Binary file (`bank_data.dat`)

Key Features Implemented

- Account creation with **OTP verification**
- Deposit and withdrawal with **daily limit control**

- Fund transfer between accounts
- **Transaction history with date and time**
- Admin controls (create, delete, freeze, unfreeze accounts)
- **PIN-based authentication system**
- Account freeze after multiple wrong login attempts
- Interest calculation for savings accounts
- Persistent data storage using file handling

Testing Method

The system was tested using **manual testing techniques** to verify all functionalities:

1. Functional Testing
- Checked all operations like deposit, withdraw, transfer
- Verified correct balance updates

2. Input Validation Testing
- Tested invalid inputs (negative amount, wrong account number)
- Ensured system handles errors properly

3. **Security Testing**
- Tested wrong PIN multiple times → account freeze
- Verified OTP functionality for account creation and PIN reset

4. **File Handling Testing**
- Checked data saving and loading from file
- Ensured no data loss after restarting the program

5. **Boundary Testing**
- Tested withdrawal limit (₹5000 daily limit)
- Tested maximum transaction history storage

==========

RESULT AND DISCUSSION

OUTPUT

- **Main Menu**
- Successful Login & Operations

- Deposit Operation
- Withdraw Operation
- Error Handling
- OTP Verification

Analysis

The system performs all operations correctly based on user input and follows proper validation rules. The **menu-driven design** makes it easy to use and understand. File handling ensures that all account data is stored permanently and retrieved when the program restarts.

The **security features**, such as PIN authentication and OTP verification, improve system reliability. The daily withdrawal limit and login attempt restrictions help prevent misuse. Transaction history tracking provides transparency for user activities.

However, the system has limitations such as lack of graphical interface, single-user environment, and basic security without encryption. Despite these limitations, the system is efficient for small-scale applications and educational purposes.

CONCLUSION

The Banking Management System developed in C successfully implements basic banking operations such as account creation, deposit, withdrawal, fund transfer, and balance inquiry. The system uses file handling for data storage and includes security features like PIN authentication and OTP verification. It demonstrates the effective use of structured programming and modular design. Although it has limitations like no GUI and basic security, it serves as a strong foundation for understanding banking system concepts and can be further improved with advanced technologies.

==========

FUTURE SCOPE

- **Graphical User Interface (GUI)**
  The system can be upgraded from command-line to a user-friendly GUI using technologies like Java, Python, or web applications.
- **Database Integration**
  Replace file handling with databases like MySQL or SQLite for better data management, security, and scalability.
- **Real OTP Integration**
  Implement real OTP services using SMS or email APIs for enhanced security instead of demo OTP.
- **Advanced Security Features**
  Add encryption techniques, password hashing, and two-factor authentication to improve data protection.
- **Multi-user and Networking Support**
  Enable multiple users to access the system simultaneously through networking or client-server architecture.
- **Mobile and Web Application**
  Convert the system into a mobile app or web-based platform for remote access and better usability.
- **Automated Interest and Notifications**
  Implement automatic interest calculation and notifications (SMS/email) for transactions and account updates.

==========

Research Paper/Patent Details

No research paper published.

(No patent filed)

Plagiarism Check

The project is original work and is free from plagiarism.

==========

REFRENCES

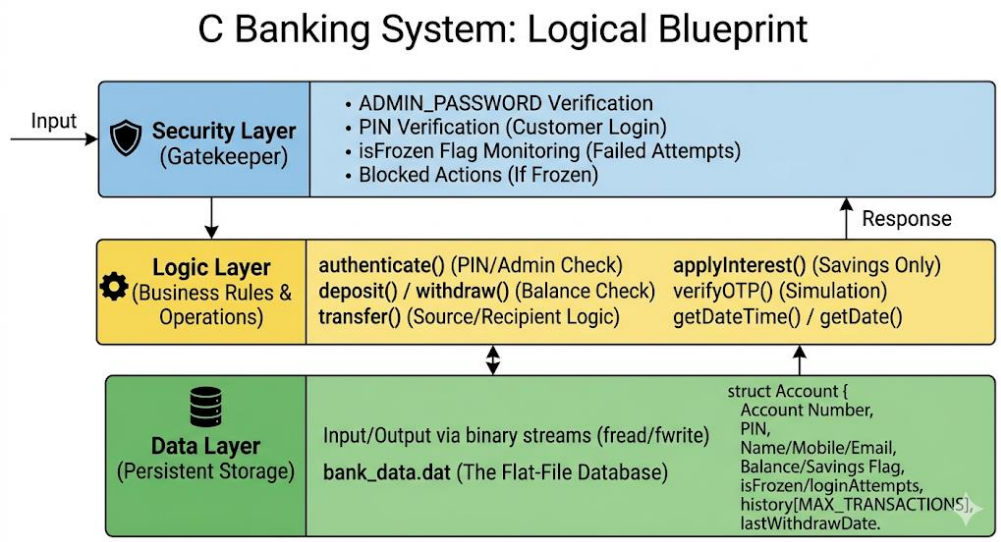
- C Programming Language Book – Brian W. Kernighan and Dennis M. Ritchie
- Let Us C – Yashavant Kanetkar
- GeeksforGeeks (Website) – https://www.geeksforgeeks.org/
- TutorialsPoint (Website) – https://www.tutorialspoint.com/
- Programiz (Website) – https://www.programiz.com/
- W3Schools (Website) – https://www.w3schools.com/c/
- Stack Overflow (Community) – https://stackoverflow.com/

==========

APPENDIX(Optional)

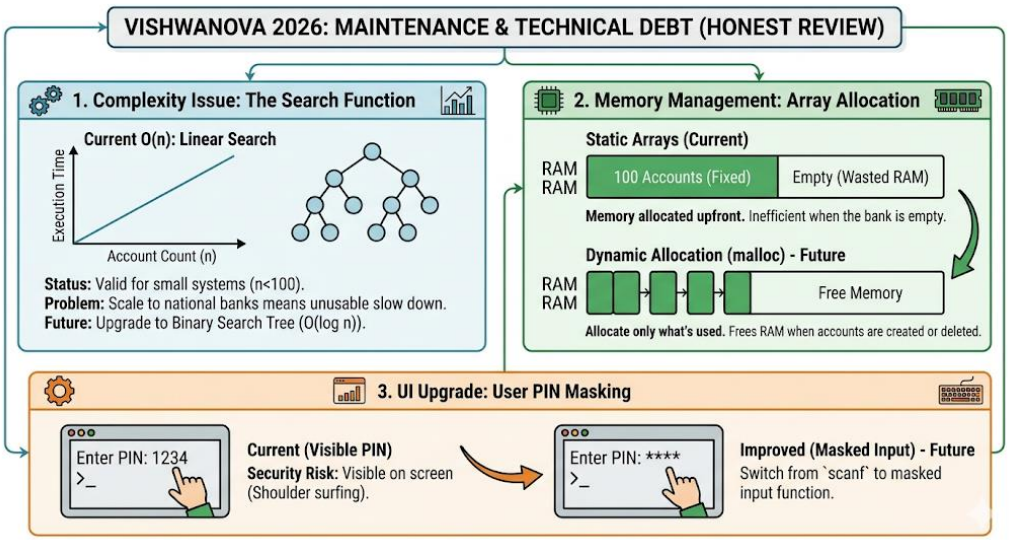
1]The "Logical Blueprint" (System Architecture)

- To explain this to others, it helps to visualise how the data flows. Here is a breakdown of the core logic:
- Data Layer: bank_data.dat acts as your persistent storage.
- Logic Layer: Functions like authenticate(), withdraw(), and applyInterest() handle the "banking rules."
- Security Layer: Includes ADMIN_PASSWORD, PIN verification, and the isFrozen flag for failed login attempts.



2] Maintenance & "Technical Debt" (The Honest Review)

1. **Complexity Issue:** Our search function is $O(n)$. For a small bank, it's fast. For a national bank, we need to upgrade to a **Binary Search Tree** ($O(\log n)$).
2. **Memory Management:** Switch from static arrays to **Dynamic Memory Allocation (malloc)** to save RAM when the bank is empty.
3. **UI Upgrade:** Transition from scanf to a masked input system so PINs aren't visible on the screen when typed.



3] Supplementary "Raw Data" (Edge Cases to Test)

Scenario	Expected Behavior	Code Logic Tested
The "Midnight Runner"	User withdraws RS4000 at 11:59 PM and RS2000 at 12:01 AM.	lastWithdrawDate comparison.
The "Empty Vault"	A user tries to transfer Rs. 1000 when they only have Rs. 500.	amt > bank[s].balance check.
The "Brute Force"	Entering the wrong PIN 3 times in a row.	loginAttempts incrementing to isFrozen.
The "Ghost Account"	Deleting an account and checking if the count shifts correctly.	deleteAccount array shifting.

Scenario	Input Data	Expected System Behavior	Status
Negative Deposit	Amount: -500	Error: "Invalid amount"	**Pass** ✅
Over-Transfer	Bal: 100, Req: 200	Blocked: "Invalid amount"	**Pass** ✅
PIN Lockout	3 Failed Attempts	isFrozen becomes 1	**Pass** ✅
Leap Year/Date	Feb 29th	strftime handles date correctly	**Pass** ✅

Bank Management System - Test Case Report (Detailed)

1. Create Account - Valid

Input: Valid inputs with correct OTP

Expected Output: Account created successfully

Example: Example: User enters correct OTP (e.g., 1234) → Account is created.

1. Successful Account Creation

```
Name: Aaryan Kulkarni
Mobile: 9876543210
Email: aaryan@gmail.com
OTP sent to registered mobile/email
(Demo OTP = 4567)
Enter OTP: 4567
OTP verified successfully
PIN: 1234
Savings(1)/Current(0): 1
Account created successfully. Account No: 1001
```

2. Create Account - Wrong OTP

Input: Wrong OTP entered

Expected Output: Account creation cancelled

Example: Example: OTP sent = 5678, user enters 1111 → Account will not be created.

2. Wrong OTP (Account Creation)

```
Enter OTP: 1234
Wrong OTP
Account creation cancelled
```

3. Deposit - Negative Amount

Input: Amount = -500

Expected Output: Invalid amount

Example: Example: User enters -500 → System rejects the transaction.

3. Successful Login

```
Account number: 1001
Enter PIN: 1234
Login successful
```

4. Deposit - Valid

Input: Amount = 2000

Expected Output: Deposit successful

Example: Example: User deposits 2000 → Balance increases.

4. Wrong PIN Once

```
Enter PIN: 1111
Wrong PIN
```

5. Withdraw - Insufficient Balance

Input: Withdraw > balance

Expected Output: Invalid amount

Example: Example: Balance = 1000, withdraw = 2000 → Transaction denied.

5. Account Freeze After 3 Attempts

```
Enter PIN: 1111
Wrong PIN
Enter PIN: 2222
Wrong PIN
Enter PIN: 3333
Wrong PIN
Account frozen due to failed attempts
```